DLDCA Lab Quiz 2

October 30, 2026

Contents

README 1

Submission Instructions 1

Q1. Assembly: String Substitution into a Bounded Buffer 3

Q2. MIPS64: Alice in Unrolling Land 5

Q3. Performance: Loop Order and SIMD for a Distance Matrix 7

Q4. Assembly: A Binary Max-Heap 10

README

This is the second lab quiz. It covers x86-64 assembly and performance. Read every instruction carefully, and note the marks for each task and subtask before starting so you can plan your time. Your code is autograded against the testcases described below, plus hidden testcases.

4 questions, 64 marks total.

If anything is unclear, trust sources in this order: **TA clarifications, then the testbench output and code comments, then this document.**

The following instructions are part of the problem statement and must be followed.

Check your tools using `before-labquiz2.pdf` before you start.

Documentation is in the lab quiz folder: the assembly and performance slides and the same cheatsheets from the weekly labs. Reference tables for assembly instructions, the ABI, syscalls and SIMD intrinsics are at the end of this document.

Good luck!

Submission Instructions

Only the files marked `(TODO)` should be edited. Leave every other file (the testbenches, the expected-output files, etc.) exactly as given.

Your submission must contain exactly the following tree:

```
your-roll-no/
|- task1/
    |- fmt_replace.s (TODO)
    |- harness.s     (provided, do not edit)
|- task2/
    |- sum.s  (TODO)
    |- loop.s (provided, do not edit)
|- task3/
    |- dist.cpp      (TODO)
    |- kernels.h     (provided, do not edit)
    |- base.cpp      (provided, do not edit)
    |- main.cpp      (provided, do not edit)
    |- reference     (provided, do not edit)
|- task4/
    |- heap.s        (TODO)
    |- main.cpp      (provided, do not edit)
```

Before building the tarball, run these from the `labquiz2` directory:

```
$ make clean
$ make check
```

`make clean` deletes stale generated files, which can cause spurious failures. `make check` runs the testcases of every task. Pass as many as you can, however passing all of them doesn't guarantee full marks.

Please read the testing and autograding sections in each task. Make sure your code compiles when you submit.

Then build the submission with:

```
$ ./build_archive.sh
Enter Student Roll Number: your-roll-no
Packaging assignment files...
  [+] Copied: ...
Successfully created your-roll-no.tar.gz
```

Do not rename the resulting tarball!

`./build_archive.sh` only builds the tarball. It does **not** submit anything. **This is the actual submission procedure:**

1. **Create a folder `~/Desktop/submission_<rollNumInLowerCase>/` and move your tarball into it**. It should be named **your-roll-no.tar.gz**!
2. **Run `check` and type your roll number** when asked. This generates your final submission from that tarball. Note your submission is finally a tarball *of a tarball*.
3. Call a TA and **Run `submit` to submit this tarball.** They will type the password :)

Q1. Assembly: String Substitution into a Bounded Buffer [10 Marks]

The task is to substitute a string into a format string without overflowing the buffer capacity. Here are the steps:

Step 1: Substitution. You get two null-terminated strings (means their last byte is literally `0`) — a template `fmt` and a replacement `arg`. Copy `fmt` character by character, but in place of every `%` write the whole of `arg`. Characters of `arg` are copied as they are, even if `arg` itself contains a `%`. Example:

fmt	arg	result
`Hello, %!`	`Bob`	`Hello, Bob!`
`%-%-%`	`ab`	`ab-ab-ab`
`a%b%c`	(empty)	`abc`
`[%]`	`50%`	`[50%]`

Step 2: Capacity. The output buffer `out` has room for `cap` bytes. The last byte must be `0`, so **at most `cap - 1` actual characters fit**. Write characters until the result is complete or `cap - 1` characters have been written, then write `0` after them. Return the number of characters written, not counting the NUL. If `cap` is `0`: write nothing and return `0`. **Each string (`fmt`, `arg`) ends with a zero byte; do not copy it.**

fmt	arg	cap	out	returned
`a%b`	`XY`	64	`aXYb`	4
`a%b`	`XY`	5	`aXYb`	4
`a%b`	`WXYZ`	4	`aWX`	3
`abcdef`	(empty)	4	`abc`	3
`x%y`	`abc`	1	(empty)	0
`x%y`	`abc`	0	not written	0

The buffer can fill up in the middle of `arg` (third row), so check the space left before every character you write.

```
if cap == 0: return 0
n = 0
for each character c of fmt:
    if c is '%':  text = arg
    else:         text = the one character c
    for each character d of text except null:
        if n == cap - 1: stop copying (leave both loops)
        out[n] = d
        n = n + 1
out[n] = 0
return n
```

Task. In `fmt_replace.s (TODO)`, write the body of

```
fmt_replace(const char* fmt{rdi}, const char* arg{rsi},
            char* out{rdx}, size_t cap{rcx}) -> size_t{rax}
```

This syntax gives the way in which any caller will call your function: first argument (`fmt`) comes in `rdi`, second argument (`arg`) comes in `rsi`, and so on. Output (number of bytes written) goes

in `rax`. You must respect caller-callee semantics: the hidden tests check that `rbx`, `rbp`, `rsp` and `r12`–`r15` are preserved. For the list of caller/callee-saved registers see the table at the end.

Do not call any library function. Do not edit `harness.s`.

`harness.s` is provided and contains `_start`. It reads three lines from stdin: `fmt`, `arg`, and `cap` in decimal (≤ 4096). It fills `out` with `#` bytes, calls `fmt_replace`, then prints the string left in `out` on the first line and the returned value on the second.

```
$ # you can run it interactively as so
$ make task1
taskset -c 1 ./task1.out
Hello, %, I am %!
Bob
64
Hello, Bob, I am Bob!
21

Return Code: 0
```

Testing

```
$ make task1-check
TCG 1 (1 mark)
  PASS -- task1/tests/01_input.txt
  PASS -- task1/tests/02_input.txt
TCG 1 PASS -- 1/1 marks
...
```

Each test is `task1/tests/NN_input.txt` (the three input lines) with the expected output in `NN_output.txt`. The tests are grouped into the testcase groups (TCG) of the table below, listed in `task1/tests/groups.txt`. A group passes only if every test in it passes.

```
  FAIL -- task1/tests/01_input.txt    see: ./task1.out < task1/tests/01_input.txt
        | diff task1/tests/01_output.txt -
```

Autograding

The following test groups are *some of what* will be used for grading. **If your code passes all tests in a group, you get the marks shown below.** Final evaluation may use additional test cases.

Group	Tests	Behaviour	Marks
1	01, 02	no `%`, one `%`	1
2	03–08	several `%`, empty `fmt` or `arg`, `%` inside `arg`	2
3	09–12	buffer full in `fmt` or in `arg`, `cap` = 1, `cap` = 0	2
4	13, 14	100 `%`s, with and without truncation	3
5	hidden		2

Hardcoding any testcases will result in 0 marks.

Q2. MIPS64: Alice in Unrolling Land [10 Marks]

This question is a MIPS64 program run in EduMIPS64. Your task is to help Alice speed up the sum of an array, $s = a[0] + a[1] + \cdots + a[N-1]$, by unrolling its loop by 4 exactly. **All cycle counts in this question are measured with forwarding off.** The pipeline rules are in Table 8 at the end of this document.

Step 1: Look at the baseline. The program `loop.s` defines the code for adding N elements of `a` (64-bit words) one per iteration. At the top, we define the labels, then there is a setup block that prepares three registers for Alice's code to work on:

- `r1`: pointer to the current element of `a`,
- `r4`: the number of elements that remain,
- `r5`: the running total, 0 at the start.

Finally, there is an end block that stores `r5` to the word `s` and stops the simulator.

Step 2: Unrolling by 4. Alice wants to write code in `sum.s` to unroll by 4 so one iteration handles elements i to $i+3$. She knows the benefit of this: the loop overhead (two `daddi`, one `bne`, one branch bubble) is now paid once per 4 elements.

Poor Alice wrote the 4 loads first then 4 adds but she noticed a problem: all 4 adds form a dependency chain.

Step 3: Multiple accumulators. Alice had a genius idea: instead of adding all into the same register, keep four separate totals (t_0, t_1, t_2, t_3) instead, one per position in the iteration: element i goes into total 0, element $i+1$ into total 1, and so on. At the end of the loop, add the four totals into `r5`.

Example. For $N = 8$ and $a = 11, 48, 85, 122, 159, 196, 233, 270$:

$$t_0 = 11 + 159 = 170, \quad t_1 = 48 + 196 = 244, \quad t_2 = 85 + 233 = 318, \quad t_3 = 122 + 270 = 392,$$

and $s = 170 + 244 + 318 + 392 = 1124$. The extra totals must start at 0.

Task. Complete `sum.s (TODO)`: write the code between the setup block and the end block. It must compute the sum with a loop unrolled by 4 and leave the total in `r5` when it reaches the end block.

- **Assume N is a multiple of 4 and $N \geq 4$. Read only `a[0]` to `a[N-1]`.**
- **Do not change the data section, the setup block or the end block.**
- Use only the instructions in Table 7.

Test benchmarks. For $N = 4096$ with forwarding off, there are three benchmarks to meet

- **14400 cycles**
- **12352 cycles**
- **11328 cycles**: **hard**. Do not spend time on it until everything else in the quiz is done.

A slack of 64 cycles is allowed.

Running

`run_edumips.sh` runs a program in EduMIPS64 and prints the cycle count and the value stored in `s`. By default, `sum.s` has a small test with $N = 8$, so `s` must be 1124:

```
$ ./run_edumips.sh task2/sum.s
forwarding: false
cycles: 46
s: 1124
```

To step through your program in the window, open it with `edumips64` and turn forwarding off in the settings first.

Use `make task2` for seeing actual sum and the cycle counts of `loop.s` and `sum.s` for $N = 8$ and $N = 4096$, without forwarding. A program unrolled as in Steps 2 and 3 gives:

```
$ make task2
program                    N        s   no forwarding
task2/loop.s               8     1124              89
task2/loop.s            4096  2041776           40969
task2/sum.s                8     1124              46
task2/sum.s             4096  2041776           14354
Run make task2-check for testcases for final grading.
```

Testing

Run `make task2-check` for checking correctness and whether your code passes the benchmarks.

```
$ make task2-check
TCG 1 (1 mark)
  PASS -- task2/sum.s N=4
  PASS -- task2/sum.s N=8
TCG 1 PASS -- 1/1 marks
...

TCG 4 (4 marks)
  FAIL -- task2/sum.s cycles 14354 > 11328 (N=4096, no forwarding)
TCG 4 FAIL -- 0/4 marks

task2: 3/10 marks
```

Autograding

If your code passes all tests in a group, you get the marks shown below.

Group	Tests	Behaviour	Marks
1	$N = 4, 8, 100, 4096$	correct sum	1
2	$N = 4096$	at most 14400 cycles, forwarding off	2
3	$N = 4096$	at most 12352 cycles, forwarding off	3
4	$N = 4096$	at most 11328 cycles, forwarding off (hard)	4

Hardcoding any testcases will result in 0 marks.

Q3. Performance: Loop Order and SIMD for a Distance Matrix [18 Marks]

Many algorithms (nearest neighbours, clustering) start from the distances between every pair of points in a set. For M points with D coordinates each, the squared distance matrix is

$$out[i][j] \;=\; \sum_{d=0}^{D-1} \big(P[d][i] - P[d][j]\big)^2 \qquad 0 \le i, j < M.$$

This takes M^2D subtractions, multiplications and additions. In this question you will speed it up.

Step 1: Memory layout. Both arrays are stored row-major in one flat block of `floats`.

Array	Shape	Element
`P`	$D \times M$: row d holds coordinate d of every point	$P[d][j]$ = `P[d * M + j]`
`out`	$M \times M$	$out[i][j]$ = `out[i * M + j]`

Increasing `j` by one moves to the next `float` (4 bytes) in both arrays. Increasing `d` by one moves `M` floats forward in `P`.

Step 2: The base version. `base.cpp` computes the matrix with the loops in the order i, j, d:

```
for i in 0 .. M-1:
    for j in 0 .. M-1:
        s = 0
        for d in 0 .. D-1:
            t = P[d*M + i] - P[d*M + j]
            s = s + t*t
        out[i*M + j] = s
```

You can add each term straight into `out[i*M + j]` instead of keeping a temporary variable `s`. The driver code sets every element of `out` to 0 before each call, so this still gives the right sum.

Step 3: Comparing loop orders. For each loop order, **analyse the memory access pattern and pick the order that gives the fewest cache misses.** This is based on concepts taught in labs. Test out your intuition by checking run time, before submitted a loop order!

Step 4: SIMD. An AVX register (type `__m256`) holds 8 floats, and one AVX instruction works on all 8 at once. **SIMD pays off when the innermost loop does the same operation on 8 elements that lie next to each other in memory, so that one load reads all of them.**

`_mm256_set1_ps(x)`	all 8 lanes set to `x`
`_mm256_loadu_ps(p)`	load the 8 floats at `p`
`_mm256_sub_ps(a, b)`, `_mm256_mul_ps`, `_mm256_add_ps`	lane-wise `a - b`, `a * b`, `a + b`
`_mm256_storeu_ps(p, v)`	store the 8 lanes of `v` at `p`

M **is a multiple of 8, so a loop over `j` in steps of 8 leaves nothing over.**

Task. In `dist.cpp (TODO)`, write

```
void dist_order(const float* P, int M, int D, float* out);   // Part 1
void dist_simd (const float* P, int M, int D, float* out);   // Part 2
```

1. **Part 1, `dist_order`:** the same computation as `dist_base`, plain scalar C++, with the three loops in the order that runs fastest.
2. **Part 2, `dist_simd`:** Extend your Part 1 with the innermost loop written in AVX intrinsics, 8 floats per step (see the intrinsics table at the end).

Do not edit `main.cpp`, `base.cpp`, `kernels.h`, `reference` or the makefile. The makefile compiles with `-O2 -fno-tree-vectorize -mavx`: the compiler optimises your code but does not vectorise loops by itself, so Part 1 stays scalar and every SIMD instruction in Part 2 comes from your intrinsics.

`main.cpp` will generate M points, computes the matrix once as reference, then runs each kernel `REPS` times. It prints the best time, `PASS` or `FAIL` against the reference, and the speedups. You can name kernels in `args` to run only those (runs all by default). With the reference solution on a lab machine:

```
$ make task3
taskset -c 1 ./task3.out
M=4096 D=64, best of 3 runs
base       2647.3 ms  PASS
order       774.0 ms  PASS
simd        186.0 ms  PASS
speedup order over base  = 3.42x
speedup simd  over order = 4.16x
speedup simd  over base  = 14.24x

Return Code: 0
```

Times vary from run to run, so run it a few times before you compare orders. The run is pinned to one core (`taskset -c 1`); keep other heavy programs closed while measuring.

Testing

`task3/reference` is the reference version of `task3.out`. `make task3-check` will run each kernel in `task3/reference` and then in your `task3.out`, on the same core, and compare your runtime. It will check for (A) correctness (B) at most 2.5× time taken as reference and (C) at most 1.3× time taken as reference. For `dist_simd` it also checks that you have used SIMD instrinsics. E.g.

```
$ make task3-check
running ./task3/reference and ./task3.out on core 1, this takes a while ...
PASS -- task3 order correct
PASS -- task3 order 210.8 ms = 1.00x reference 211.4 ms <= 2.5x
PASS -- task3 order 210.8 ms = 1.00x reference 211.4 ms <= 1.3x
PASS -- task3 simd correct
PASS -- task3 simd 61.4 ms = 1.01x reference 61.0 ms <= 2.5x
PASS -- task3 simd 61.4 ms = 1.01x reference 61.0 ms <= 1.3x
Times on lab machines... Final grading runs every submission on one machine.
```

Autograding

Results of `make task3-check` on a lab machine are indicative only: please run many times and ensure no other heavy processes are running. For grading, every submission will be run on the same machine. Still, the below benchmarks will give you a good idea about how much marks you will actually score.

Group	Kernel	Behaviour	Marks
1	`dist_order`	`PASS`	2
2	`dist_order`	best time at most 2.5 × reference	2
3	`dist_order`	best time at most 1.3 × reference	4
4	`dist_simd`	`PASS`, with SIMD arithmetic instructions	3
5	`dist_simd`	best time at most 2.5 × reference	3
6	`dist_simd`	best time at most 1.3 × reference	4

Hardcoding any testcases will result in 0 marks.

Q4. Assembly: A Binary Max-Heap [26 Marks]

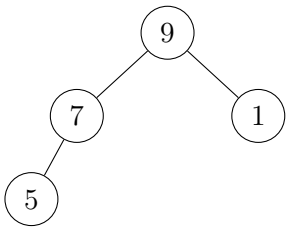
Step 1: Remember max-heap? A priority queue hands out its largest element first. A *max-heap* is a data structure that implements one. It supports two operations: *push* adds an element, and *pop* removes and returns the largest element. Both take time proportional to $\log_2 n$ for n elements.

A max-heap is a binary tree in which every node is greater than or equal to its children. The largest element is therefore always at the root. There is no ordering between the two children of a node, and none between nodes on different branches.

We store the tree in an array, level by level, so the root is `data[0]`, its children are `data[1]` and `data[2]`, and so on. For the node at index i:

parent	$(i-1)/2$ (integer division, for $i > 0$)
left child	$2i+1$
right child	$2i+2$

A child index $\geq$ `size` means that child does not exist. The tree and the array below hold the same heap:



`data = [9, 7, 1, 5]`, `size = 4`

Step 2: The heap object. `main.cpp` defines the heap as a C++ class with three fields. You implement two of its operations, `push_heap` and `pop_heap`, in assembly. Each receives a pointer to the object in `rdi`.

```
class heap {
public:
    int64_t* data;     // the array, allocated by C++
    size_t   size;     // number of elements in the heap
    size_t   capacity; // number of elements data can hold
    ...
};
```

Assembly has no field names. You must read and write each field at its byte offset from the object pointer, so check the offsets before writing any `mov`:

Offset	Field	Who changes it
0	`int64_t* data`	C++ only
8	`size_t size`	your `push_heap` and `pop_heap`
16	`size_t capacity`	C++ only

The elements are `int64_t`, 8 bytes each, so `data[k]` is at address `data + 8*k`. They are *signed*: compare them with `jl`, `jle`, `jg`, `jge`. The unsigned jumps (`jb`, `ja`, ...) treat a negative number as a very large one.

Before `push_heap` is called, `size < capacity`, so `data[size]` is free. Before `pop_heap` is called, `size > 0`. Your functions never allocate memory and never call another function.

Step 3: Push (sift-up). Put the new value in a hole at the end and move the hole up: while the parent of the hole is smaller than the value, move the parent down into the hole.

```
push_heap(h, val):
    i = size
    size = size + 1
    while i > 0 and data[(i-1)/2] < val:
        data[i] = data[(i-1)/2]
        i = (i-1)/2
    data[i] = val
```

Pushing 5, 9, 1, 7 into an empty heap gives `[5]`, `[9, 5]`, `[9, 5, 1]`, `[9, 7, 1, 5]`. For the last push the hole starts at index 3; its parent `data[1] = 5` is smaller than 7 and moves down, then the parent `data[0] = 9` is not, so 7 goes into index 1.

Step 4: Pop (sift-down). The root is the value to return. The last element must find a new place, and the slot it leaves, `data[n]` with `n = size - 1`, now lies outside the heap. **Store the popped value in `data[n]`.** Then move a hole down from the root: while the larger child of the hole is bigger than the old last element, move that child up into the hole.

```
pop_heap(h):
    n = size - 1
    size = n
    top = data[0]
    last = data[n]
    data[n] = top
    i = 0
    loop:
        c = 2*i + 1                          # left child
        if c >= n: stop
        if c + 1 < n and data[c+1] > data[c]:
            c = c + 1                        # right child is larger
        if data[c] <= last: stop
        data[i] = data[c]
        i = c
    data[i] = last
    return top
```

When the two children are equal, the left child is used. Popping `[9, 7, 1, 5]` gives `top = 9`, `last = 5`, `n = 3`; 7 is the larger child of the root and moves up, the hole at index 1 has no child below index 3, so 5 goes there: `data = [7, 5, 1, 9]` with `size = 3`. The 9 at `data[3]` is what `std::pop_heap` also leaves behind. If you pop every element, the array ends up sorted from smallest to largest; this is heapsort.

Task. In `heap.s (TODO)`, write

```
push_heap(heap* h{rdi}, int64_t val{rsi})
pop_heap(heap* h{rdi}) -> int64_t{rax}
```

Follow the pseudocode exactly. The tests compare the whole array after each operation, so a different but still valid heap fails. Update `size`; do not change `data` or `capacity`. Write only inside the heap; for pop, `data[n]` counts as inside. `data` may point to a new array between

calls, so load it from `h` on every call. If you use a callee-saved register, save and restore it with `push`/`pop`: the compiled C++ code keeps its own values there. **Do not edit `main.cpp`.**

`main.cpp` reads one command per line from stdin and prints a line for `pop` and `show`:

Command	Effect
`push V`	push the signed 64-bit value `V`
`pop`	pop and print `pop V`, or `pop empty`
`show`	print `size N:` and `data[0] ...data[N-1]`

After each call it also checks what the call changed and prints an `ERROR` line if `size` did not go up or down by one, if `data` or `capacity` changed, if `data[size]` is not the popped value after a pop, or if an element outside the heap was overwritten.

```
$ make task4
taskset -c 1 ./task4.out
push 5
push 9
push 1
push 7
show
size 4: 9 7 1 5
pop
pop 9
show
size 3: 7 5 1
(Ctrl+D ends the input)

Return Code: 0
```

Testing

```
$ make task4-check
TCG 1 (2 marks)
  PASS -- task4/tests/01_input.txt
  PASS -- task4/tests/02_input.txt
TCG 1 PASS -- 2/2 marks
...
TCG 8 (5 marks)
  Hidden testcases missing, will be present on grader's setup.
TCG 8 not run -- ?/5 marks

task4: 21/21 marks from the groups that ran
```

Each test is a list of commands in `task4/tests/NN_input.txt` with the expected output in `NN_output.txt`. The tests are grouped into the testcase groups (TCG) of the table below, listed in `task4/tests/groups.txt`. A group passes only if every test in it passes. A failing test prints the `diff` command that shows where your output differs.

Autograding

The following test groups are *some of what* will be used for grading. **If your code passes all tests in a group, you get the marks shown below.** Final evaluation may use additional test cases.

Group	Tests	Behaviour	Marks
1	01, 02	push on an empty heap, pop of a single element	2
2	03–05	push only	3
3	06–09	pop only	4
4	10–13	push and pop mixed	4
5	14, 15	edge case handling: equal values (ties), negative values	2
6	16–19	full heapsort: push many values, pop all	4
7	20, 21	growth beyond the first capacity	2
8	hidden		5

Hardcoding any testcases will result in 0 marks.

Table 1: Some basic x86-64 Assembly Instructions

Instruction	Operands	Effect
`mov`	`reg, reg/*mem/imm`	`reg` ← `reg/*mem/imm`
`mov`	`*mem, reg`	`*mem` ← `reg`
`movzx / movsx`	`reg, reg/*mem`	`reg` ← `reg/*mem`, zero-/sign-extended
`lea`	`reg, *mem`	`reg` ← address of `*mem`
`add`	`reg, reg/*mem/imm`	`reg` ← `reg` + `reg/*mem/imm`
`add`	`*mem, reg/imm`	`*mem` ← `*mem` + `reg/imm`
`sub`	`reg, reg/*mem/imm`	`reg` ← `reg` − `reg/*mem/imm`
`sub`	`*mem, reg/imm`	`*mem` ← `*mem` − `reg/imm`
`inc`	`reg/*mem`	`reg/*mem` ← `reg/*mem` + 1
`dec`	`reg/*mem`	`reg/*mem` ← `reg/*mem` − 1
`neg`	`reg/*mem`	`reg/*mem` ← −`reg/*mem` (Two's complement)
`not`	`reg/*mem`	`reg/*mem` ← ¬`reg/*mem` (One's complement/bitwise)
`shl / shr`	`reg/*mem, imm/cl`	Logical shift left / right by `imm` or `cl` bits
`sar`	`reg/*mem, imm/cl`	Arithmetic shift right (preserves sign bit)
`imul`	`reg, reg/*mem/imm`	`reg` ← `reg` × `reg/*mem/imm`
`imul`	`*mem, reg/imm`	`*mem` ← `*mem` × `reg/imm`
`div`	`reg`	`rax` ← `(rdx:rax)`/`reg` `rdx` ← `(rdx:rax)`%`reg`
`and`	`reg, reg/*mem/imm`	`reg` ← `reg` ∧ `reg/*mem/imm`
`and`	`*mem, reg/imm`	`*mem` ← `*mem` ∧ `reg/imm`
`or`	`reg, reg/*mem/imm`	`reg` ← `reg` ∨ `reg/*mem/imm`
`or`	`*mem, reg/imm`	`*mem` ← `*mem` ∨ `reg/imm`
`xor`	`reg, reg/*mem/imm`	`reg` ← `reg` ⊕ `reg/*mem/imm`
`xor`	`*mem, reg/imm`	`*mem` ← `*mem` ⊕ `reg/imm`
`cmp`	`reg/*mem, reg/imm`	Sets flags based on subtraction: `reg/*mem` - `reg/imm`. ZF = 1 if equal, SF = 1 if result negative, CF = 1 if borrow (unsigned), OF = 1 if signed overflow.
`test`	`reg/*mem, reg/imm`	Sets flags based on bitwise AND: `reg/*mem` ∧ `reg/imm`. ZF = 1 if result is zero, SF = 1 if high bit set, CF and OF are cleared. No result is stored.
`jmp`	`label`	Unconditional jump to label
`je / jne`	`label`	Jump if (not) equal / zero
`ja / jae`	`label`	Jump if above / above-or-equal (unsigned)
`jb / jbe`	`label`	Jump if below / below-or-equal (unsigned)
`jg / jge`	`label`	Jump if greater / greater-or-equal (signed)
`jl / jle`	`label`	Jump if less / less-or-equal (signed)
`cmov`*cc*	`reg, reg/*mem`	`reg` ← `reg/*mem` if *cc* holds, else unchanged.
`set`*cc*	`reg/*mem`	`reg/*mem` ← 1 if *cc* holds, else 0.
`syscall`		Performs a syscall with syscall number stored in `rax` Arguments passed in other registers starting from `rdi`
`push`	`reg`	Pushes the value in `reg` onto the stack
`pop`	`reg`	Pops the value on top of the stack onto `reg`
`call`	`label`	Calls the function called `label`
`ret`		Pops the return address from stack and jumps to it

Table 2: System V AMD64 ABI — Calling Convention Quick Reference

Rule	**Detail**
Argument order	`rdi, rsi, rdx, rcx, r8, r9` (1st through 6th argument)
Return value	`rax` (for a 64-bit return value)
Caller-saved (volatile)	`rax, rcx, rdx, rsi, rdi, r8, r9, r10, r11`
Callee-saved (preserved)	`rbx, rbp, r12, r13, r14, r15, rsp`
Float arguments, return	`xmm0`–`xmm7` in order; return in `xmm0`
SIMD registers	all `xmm`/`ymm` registers are caller-saved
Raw `syscall` argument order	`rax` = syscall number, then `rdi, rsi, rdx, r10, r8, r9`
More rules	Before a `call` to a libc function, ensure the stack is 16-byte aligned. One way: `and rsp, -16` at the start of your function, then maintain alignment through matched `push`/`pop` pairs.

Table 3: Limited Syscall Table

Syscall	`rax`	Arguments (`rdi, rsi, rdx`)	Returns (`rax`)
`read`	0	`fd, buf, count`	bytes read (0 = EOF)
`write`	1	`fd, buf, count`	bytes written
`exit`	60	`status`	does not return

Table 4: SSE and AVX intrinsics (`<immintrin.h>`)

SSE (4 floats, `__m128`)	AVX (8 floats, `__m256`)	Effect
`_mm_set1_ps(x)`	`_mm256_set1_ps(x)`	every lane = `x`
`_mm_setzero_ps()`	`_mm256_setzero_ps()`	every lane = 0
`_mm_loadu_ps(p)`	`_mm256_loadu_ps(p)`	load from `p`
`_mm_storeu_ps(p, v)`	`_mm256_storeu_ps(p, v)`	store `v` to `p`
`_mm_load_ps(p)`	`_mm256_load_ps(p)`	like `loadu`; `p` 16- / 32-byte aligned
`_mm_store_ps(p, v)`	`_mm256_store_ps(p, v)`	like `storeu`; `p` 16- / 32-byte aligned
`_mm_add_ps(a, b)`	`_mm256_add_ps(a, b)`	lane-wise `a + b`
`_mm_sub_ps(a, b)`	`_mm256_sub_ps(a, b)`	lane-wise `a - b`
`_mm_mul_ps(a, b)`	`_mm256_mul_ps(a, b)`	lane-wise `a * b`
`_mm_div_ps(a, b)`	`_mm256_div_ps(a, b)`	lane-wise `a / b`
`_mm_min_ps(a, b)`	`_mm256_min_ps(a, b)`	lane-wise minimum
`_mm_max_ps(a, b)`	`_mm256_max_ps(a, b)`	lane-wise maximum
`_mm_sqrt_ps(a)`	`_mm256_sqrt_ps(a)`	lane-wise square root
`_mm_cvtss_f32(v)`	`_mm256_cvtss_f32(v)`	the float in lane 0

The makefile compiles with `-mavx`. Do not add `-mavx2`, `-mfma` or `-march=native`: some lab machines do not support the instructions they produce, so `_mm256_fmadd_ps` and the 256-bit integer intrinsics (`_mm256_add_epi32`, ...) are not available.

Table 5: `perf stat` events

Event	Counts
`task-clock`	milliseconds the program ran on the CPU
`cycles`	clock cycles of the core
`instructions`	instructions that finished
`branches`	branch instructions executed
`branch-misses`	branches that were predicted wrong
`L1-dcache-loads`	reads from the L1 data cache
`L1-dcache-load-misses`	reads that were not in L1

IPC is `instructions / cycles`. On laptops with two kinds of cores, `perf` prints one row per kind (`cpu_core`, `cpu_atom`); read the row of the core you pinned to.

Table 6: Common `perf` commands

Command	Does
`perf list`	list the events this CPU can count
`perf stat -e EV1,EV2 ./prog`	count events over the whole run
`perf stat -r 5 -e EV ./prog`	run 5 times, print the mean and spread
`perf record -e EV ./prog`	sample where events happen, save to `perf.data`
`perf report -stdio`	functions from `perf.data`, most samples first
`perf annotate -stdio 'f()'`	assembly of `f` with samples per instruction

Put `taskset -c 1` in front of `perf stat` and `perf record` to pin the run.

Table 7: MIPS64 instructions and directives for Q2 (EduMIPS64)

Instruction	Operands	Effect
`ld`	`rt, off(rs)`	`rt` ← 64-bit value at address `rs + off`
`sd`	`rt, off(rs)`	64-bit value at address `rs + off` ← `rt`
`dadd`	`rd, rs, rt`	`rd` ← `rs` + `rt`
`dsub`	`rd, rs, rt`	`rd` ← `rs` − `rt`
`daddi`	`rt, rs, imm`	`rt` ← `rs` + `imm`
`dsll`	`rd, rt, sa`	`rd` ← `rt` shifted left by `sa` bits
`slt`	`rd, rs, rt`	`rd` ← 1 if `rs` < `rt` (signed), else 0
`slti`	`rt, rs, imm`	`rt` ← 1 if `rs` < `imm` (signed), else 0
`beq / bne`	`rs, rt, label`	jump to `label` if `rs` = / ≠ `rt`
`j`	`label`	jump to `label`
`nop`		do nothing (still takes a pipeline slot)
`syscall 0`		end the program
`.data / .text`		start the data / instruction section
`.word`	`v1, v2, ...`	store 64-bit values
`.space`	`n`	reserve `n` bytes (zeroed)

A label is a name followed by a colon. A comment starts with `;`. Registers are `r0`–`r31`; `r0` is always 0. The instruction after a branch runs only if the branch is not taken.

Table 8: Pipeline rules of the EduMIPS64 model (five stages: IF, ID, EX, MEM, WB)

Situation	Forwarding off	Forwarding on
Cycle count	4 + instructions executed + stall cycles + branch bubbles	
Consumer right after its producer (`dadd` then `dadd`)	2 stall cycles	0
One instruction in between	1 stall cycle	0
Two or more in between	0	0
`ld`, then the next instruction uses the loaded register	2 stall cycles	1
Register read by `sd` (the value it stores) or by a branch	same as any consumer: the read happens in ID	same, except: an `sd` storing a just-loaded value costs 0; a branch right after an ALU instruction (`dadd`, `dsub`, `daddi`) that computes its register costs 1; a branch right after an `ld` costs 2, and with one instruction in between costs 1
Branch taken	1 bubble: the instruction fetched behind it is thrown away	
Branch not taken	0	

The branch is decided in ID. There is no branch predictor: fetch continues with the next sequential instruction, which is discarded if the branch is taken. "Producer" means the instruction that writes the register; "in between" counts instructions between the producer and the consumer. Every program here ends with `syscall 0`, and the count runs until it leaves WB.